Exactly Verifiable Infeasibility Certificates for Robot Motion Planning,

with a Verifier Proved Sound in Lean

Stéphane Cambon

Cambon AI, Paris, France

Version 2, 12 October 2026

Abstract

When a motion planner fails to find a path, it produces no evidence that none exists; yet task-and-motion planners pruning a search, and safety engineers arguing that a robot cannot reach an operator zone, need a *proof*. We present infeasibility certificates for motion planning of articulated arms that an independent program can check without trusting the planner, the LP solver or any collision checker. The check is small and exact—499 lines of standard-library Python in rational arithmetic—but exactness is the means, not the point: the proof stands on its own. The method is sound and incomplete: cells of the joint box it cannot certify are reported as undecided, but it never produces a false certificate; no approximate quantity enters the theorem. The theorem quantifies over every joint of the arm; only the cost is governed by the number of *active* dimensions of the trapped body—the joints its geometry depends on—and on a synthetic family we certify up to seven active dimensions. Certificates are slab-aware witnesses from linear programs over Bernstein coefficients on a rational parametrization of forward kinematics; no semidefinite program enters the trust chain. On a laptop CPU we certify a KUKA iiwa7 whose kinematics reproduce the published URDF to $2 \cdot 10^{-6}$ m and whose trapped link carries its faithful convex silhouette, unable to reach a target behind an overhead shelf, in under a minute; the verifier re-checks it in 2.4 s. A second verifier, in Lean 4, is proved sound in Lean/Mathlib under the three standard axioms—if it accepts, no collision-free path exists—and agrees with the first on every shipped certificate, an adversarial corpus and a thousand fuzzed inputs; for one scene the certificate is evaluated in the kernel and its disconnection is a theorem. We also report two cases where dense sampling vouched for a disconnection that did not exist and the certifier refused.

1 Introduction

Completeness is the property motion planning most often gives up. A complete planner returns a path when one exists and reports non-existence otherwise; in high-dimensional configuration spaces, practical planners return paths when they can and time out when they cannot. The asymmetry matters because non-existence is not a corner case. Task-and-motion planning (TAMP) explores alternatives whose motion subproblems are frequently infeasible, and treats a timeout as a weak, expensive signal [3, 8, 11]. Safety cases for collaborative cells must argue that an arm cannot reach a protected zone under any joint configuration. Workcell and fixture design must establish that a target is unreachable *before* the cell is built. In all three, the desired output is a proof, and the proof must be one the consumer can check.

Recent work has made infeasibility proofs practical for manipulators. Li and Dantam construct a separating manifold in the obstacle region of the configuration space—first by learning, later by Coxeter triangulation on a GPU—and prove that it separates start from goal [19, 21–23]. Henrion, Miller and Safey El Din prove path-disconnectedness of abstract semialgebraic sets with a moment–sum-of-squares hierarchy [14]. Dai, Amice and co-authors certify collision-*free* polytopes in a rational parametrization of the configuration space [7]. These results share a property that is easy to overlook: the certificate's validity rests on a floating-point computation. A triangulated manifold is declared inside the obstacle region by a numerical collision checker; the disconnectedness and free-space certificates are semidefinite programs' outputs, accepted at solver tolerance. (Rationalizing a sum-of-squares certificate to exact arithmetic is possible and is an active line of work [12, 25, 31], but it has not been applied to these certificates, and it hinges on projecting the solver's Gram matrix onto an exactly positive-semidefinite one—a step that works on small, structured instances and is not routine at scale.) The consumer trusts the pipeline. For pruning a TAMP search that is a reasonable bargain. For a safety argument, or for a reviewer who wants to recount the proof, it is not what one would want.

Our contribution is not a higher degree-of-freedom count; it is a change in the nature of the certificate. We produce infeasibility certificates for articulated arms that an independent program re-verifies in exact rational arithmetic, from the certificate alone. The verifier is 499 lines of Python that import nothing beyond the standard library and nothing from the generator. It re-derives the forward kinematics from the joint chain declared in the certificate, re-forms the algebraic positivity conditions itself, and re-proves that the certified cells tile the joint box. An auditor can read it in an afternoon and recount the proof by hand on a small instance.

Rational arithmetic is not the contribution; it is what makes the checker small enough to trust. The contribution is that the check depends on nothing the generator did: this is the Exact Geometric Computation principle of Yap [41]—a geometric decision must never be wrong—applied to a proof object rather than to an algorithm. In this version the independence of the check is itself established by a proof assistant: a second verifier, written in Lean 4, is proved sound against reference definitions of the kinematics and of collision (§6).

Two things a first reading tends to miss, so we state them here. First, the method is *sound and incomplete*. The joint box is bisected into exact rational cells; on each cell a conservative test either certifies collision or does not. A cell that cannot be certified is reported as undecided and no certificate is issued; a certified cell is checked exactly. The decomposition is therefore a source of incompleteness, never of error, and no approximate quantity enters the theorem (§4). Second, the theorem quantifies over *every* joint of the arm. The number of *active* dimensions of a trapped body—the joints its geometry depends on—governs the size of each cell's LP, not the scope of what is proven; on a synthetic family the active count is raised from 3 to 7 with every instance certified and exactly verified (§7.2).

The method (§3) rests on three ingredients, none of them new in isolation. First, the half-angle substitution $s = \tan((q - q^*)/2)$ makes the forward kinematics of a revolute chain a rational function with a common denominator per link; this is the parametrization used by C-IRIS [7] and available in Drake [35]. Second, a low-degree scalar barrier φ defines a slab $\{|\varphi| \le \delta\}$ in joint space; if the slab lies inside the obstacle region and start and goal lie strictly on opposite sides, no path connects them. Third, on each cell of a branch-and-bound partition, a *witness point* moving with the robot—an affine combination of the vertices of a convex body, with coefficients that are themselves polynomials in s—is shown to lie inside an obstacle polytope on the whole cell $\cap$ slab, by a Putinar-style product $g - \mu T$ whose non-negativity is established through Bernstein coefficients. Every constraint is linear in the unknowns, so each cell is an LP; no semidefinite program appears in the trust chain, and making the solution exact is a matter of rounding rational coefficients and re-checking inequalities—no positive-semidefinite matrix has to be recovered. The only floating-point computation, the LP solve, is downstream of nothing the verifier trusts.

The contributions are:

(C1) A certificate schema for configuration-space disconnection of articulated arms—slab-aware Bernstein-LP witnesses over a rational FK parametrization—with a three-valued verdict: PROOF (exactly verified), ENGINE-PROOF (accepted by the generator, exact verification unavailable for the scene class, always flagged), and UNDECIDED, which is never read as "infeasible".

(C2) An independent exact verifier (< 500 lines, standard library only, zero imports from the generator) that re-derives kinematics, re-forms the positivity products, re-proves the tiling, and is hardened against 32 adversarial certificate mutations—26 on a planar certificate, 6 on a locked-joint certificate—together with a spatial mutation suite: moved locked joints, corrupted rational rotations, removed leaves (holes), duplicated leaves (overlaps), among others.

(C3) A measured cost model: the per-leaf LP grows as $(d+1)^k$ in the number k of *active* dimensions of the certified body, while passive (redundant) dimensions are covered by the theorem at almost no cost. On the 7-DOF flagship the active-dimension reduction is $443\times$; on a family of sealed synthetic disconnections the affine witness certifies every k from 3 to 7, with the practical frontier set by the size of a single LP rather than by certifiability.

(C4) Certified scenes from 2-DOF regression cases to a KUKA iiwa7 with kinematics faithful to the published URDF ($2 \cdot 10^{-6}$ m at the flange) and a faithful convex link silhouette (40 support vertices, $1.7 \cdot 10^{-6}$ m), certified in under a minute on a laptop CPU and verified in 2.4 s; three use-case scenes (bin picking, pharmacy shelf, safety guard) on the same robot; and two documented episodes in which the certifier caught disconnection claims that dense sampling had validated.

(C5) A verifier in Lean 4 whose soundness is a machine-checked theorem (19 proof obligations, 145 theorems, three standard axioms, no `sorry`), stated against reference definitions that contain no polynomial; measured agreement with the Python verifier on 13 certificates, a 48-case adversarial corpus and 1 000 fuzzed inputs with no divergence; and, for one shipped scene, a disconnection that is a theorem of Lean/Mathlib, the checker's acceptance being evaluated in the kernel. The proof also sorts the verifier's conditions into those soundness needs, those that give the statement its meaning, and those that merely let the verifier follow the generator (§6.3).

Scope, stated up front. The theorem quantifies over a box of joint values with $|q_i - q_i^*| < \pi$ (no wrap-around), for serial revolute chains with $q^* = 0$ or locked joints at exactly rational $(\cos, \sin)$; obstacles are static polytopes in H-representation with rational coefficients; certified bodies are convex hulls of rational vertices; the barrier φ is supplied with the scene (an automatic fitting pipeline exists and is validated on planar scenes, but the certificates we ship use scene-provided barriers). The kinematic model is exact as an internal object; its fidelity to the physical robot is bounded by the published URDF, whose joint axes are quaternion-rounded to about $3.7 \cdot 10^{-6}$ rad—we certify a rational kinematics faithful to the published description to $2 \cdot 10^{-6}$ m at the flange, not "the exact iiwa" in an absolute sense, since no such description exists. Finally, the disconnections our scheme certifies at low cost are *proximal*: a body early in the chain is trapped and no setting of downstream joints frees it. This is a selection effect of the method—a low-degree scalar barrier and a slab—and not a property of disconnections in general; it happens to match the disconnections that matter in practice (bins, shelves, guards), and we state it as a scope rather than hide it (§7).

A note on provenance: the TAMP motivation above closes a loop with the author's own early work on hybrid task and motion planning [3], cited as such by Li and Dantam [23].

The present work was carried out with AI coding assistants under human direction; every gate, scene validation and design decision carries a human signature, and the verifier is the trust anchor precisely because the generation pipeline is not to be trusted. We return to this in the provenance note (§10).

2 Related Work

Infeasibility proofs for manipulators. Li and Dantam's line of work is the closest in aim. Starting from a general framing [18], they learn a closed separating manifold in the obstacle region with an SVM alongside a bidirectional RRT, triangulate it, and check containment in the obstacle region numerically; proofs for 3- and 4-DOF arms take a few minutes on a CPU ("less than 4 minutes on average" for a 4-DOF SCARA) [19, 21]. Convergence guarantees under sampling assumptions follow in [20]. Coxeter triangulation of the manifold [22] and its batched GPU implementation [23] bring the approach to 5- and 6-DoF scenes, "both 6-DoF scenes taking less than a minute on average" on an RTX 4070 with an i9-13900K. Their certificate is a triangulated manifold whose obstacle containment is asserted by a floating-point collision pipeline; ours is an algebraic object checked in exact arithmetic by an independent program. We make no speed comparison: the scenes, hardware and proof objects differ, and the scenes we certify exploit passivity that theirs may not. What we do claim is that we reach the same degree-of-freedom frontier on a laptop CPU, with a certificate of a different nature.

Algebraic disconnection. Henrion, Miller and Safey El Din prove path-disconnectedness of two subsets of a basic semialgebraic set with time-dependent barrier functions and the moment–SOS hierarchy [14]. Their scheme is necessary and sufficient, on abstract sets, with examples up to three variables, and its certificates are semidefinite-program outputs in floating point. We certify disconnections in up to seven active dimensions, in a different setting: our scheme is sufficient-only, specialized to articulated arms, and exactly re-verifiable. The two are complementary—theirs answers the abstract question completely in low dimension; ours answers a restricted question at scale, with an auditable object. Exact symbolic connectivity queries also exist for robot kinematics, for cuspidal robots and singularity-free paths [5, 6]; that is a different question—the connectivity of the kinematic map's domain, with no workspace obstacles.

Exact certificates of positivity. That a sum-of-squares certificate can be made exact is not new: Peyrl and Parrilo round the numerical Gram matrix and project it back onto the affine constraints [31]; Kaltofen et al. refine by high-precision Newton iteration [17]; Magron and Safey El Din's RealCertify library implements rounding–projection and perturbation–compensation schemes for Putinar-type representations [24–26]; Henrion's recent notes construct rational moment and SOS certificates for classical examples in exact arithmetic [12, 13], and a 2026 preprint ships an exact rational SOS certificate with archived verification scripts [32]. Two things distinguish our setting. First, the fragile step of that line—recovering an exactly positive-semidefinite Gram matrix from an interior-point solution, which requires strict feasibility and degrades with size—does not exist here: our certificates are linear programs whose constraints are nonnegative Bernstein coefficients, so exactness is rounding rationals and re-checking inequalities, and it has succeeded at the first denominator bound on every scene we ship. Second, the exact-SOS line certifies individual polynomials; we certify the disconnection of a 7-DOF arm's configuration space, in which exactness must hold across a partition, a kinematic chain and a barrier at once. The authors of [14] state our motivation in their own words—existing algorithms "do not provide a certificate that can be checked a posteriori in the disconnected case"—and we regard the two approaches as aiming at the same target from different sides.

Exact geometric computation. The principle that a geometric decision must never be wrong, whatever arithmetic it costs, is the Exact Geometric Computation paradigm of Yap [41, 42], realised in the exact kernels of LEDA and CGAL. Our verifier is that paradigm applied to a proof object rather than to an algorithm: the generator may compute however it likes, in floating point and with heuristics, because every decision that enters the theorem is re-taken exactly by a program that trusts none of it. Rationalizing is the cheap way to be exact here, not the point of the exercise.

Machine-checked positivity. Proving polynomial inequalities inside a proof assistant has its own line: Muñoz and Narkawicz formalised Bernstein polynomials and their range-enclosure property in PVS and used them for global optimisation [30]; Solovyev and Hales verified the nonlinear inequalities of the Flyspeck project in HOL Light with Taylor interval approximations [34]; Magron, Allamigeon, Gaubert and Werner interfaced template-based certificates with Coq [27]; and Martin-Dorel and Roux's ValidSDP checks sum-of-squares certificates from a numerical solver by a reflexive Coq tactic [28]. Our Lean development (§6) belongs to this family by its Bernstein lemma, but what it certifies is not a polynomial: it is the disconnection statement of Theorem 1, kinematics and partition included, and the reference definitions it is proved against are the geometric ones. Verified certification of unsolvability exists in AI planning—Wang and Abdulaziz verify in Isabelle/HOL a certifier for temporal planning problems [40]—but, after a limited search, we found no machine-checked verifier of infeasibility or disconnection certificates for geometric motion planning.

Certified free space. C-IRIS [1, 7] answers the complementary question: it grows convex polytopes in a rational parametrization of the configuration space and certifies them collision-free with sums of squares. We work in the same rational parametrization—indeed we call Drake's

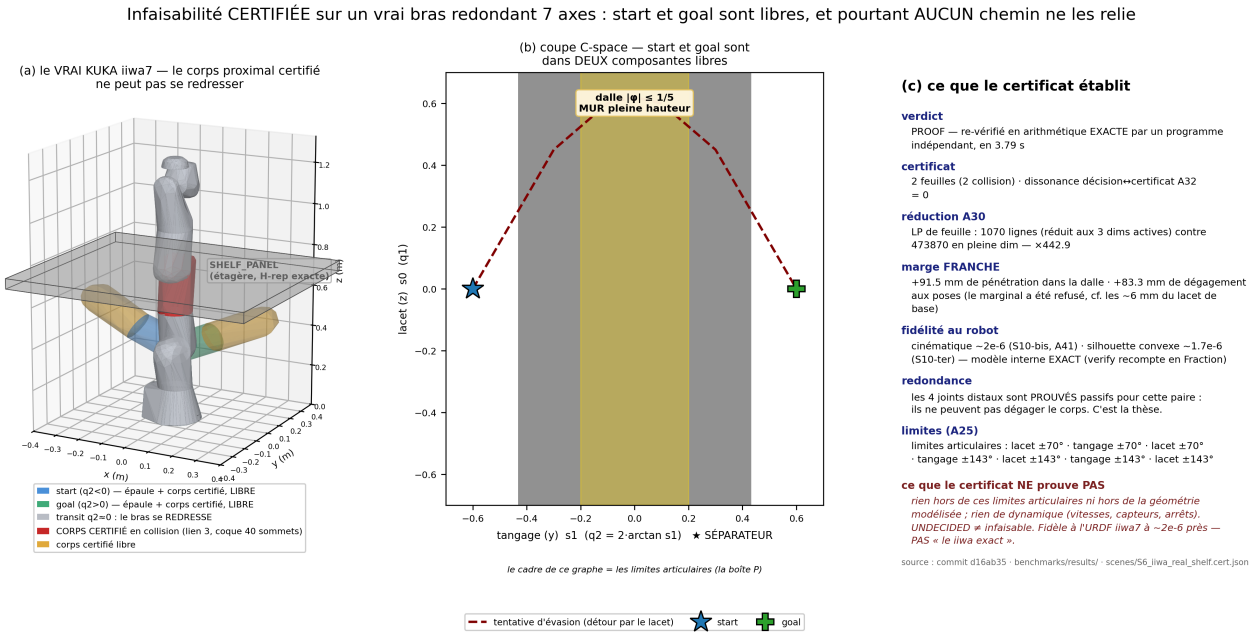


Figure 1: The flagship certificate. A KUKA iiwa7 with kinematics faithful to the published URDF (parity $1.99 \cdot 10^{-6}$ m at the flange over 1 500 configurations) and the faithful 40-vertex convex silhouette of link 3 (highlighted; the only body in the certified collision pair). Left: start and goal poses (arm inclined $\mp 61.9°$, link top at 0.597 m) pass beneath an overhead shelf whose underside is at 0.68 m; the upright transit (link top 0.808 m) strikes it. Centre: configuration-space cut along shoulder pitch q_2 with the collision wall spanning the full height and the slab $|\varphi| \leq 1/5$ inside it. Right: certificate statistics read from the dated benchmark file—PROOF, 2 leaves, active dimensions (q_1, q_2, q_3), reduced LP 1 070 rows vs 473 870 full (443×), verify 2.42 s; separation margins 91.5 mm / 83.3 mm.

`RationalForwardKinematics`, which originates in that work—and we owe the observation that revolute kinematics become rational under the half-angle substitution to that line. We differ in the question (disconnection rather than free regions), in the certificate (linear rather than semidefinite programming, so that verification reduces to exact rational arithmetic), and in providing an independent verifier. C-IRIS has been demonstrated on a 7-DOF iiwa; so has our method.

Path non-existence, numerical. Proving that no path exists by labelling configuration-space cells [43], by alpha shapes over samples [29], by caging analysis of rigid bodies [38], or by incremental bitmap segmentation up to 5 DOF [37] gives a detection, in floating point and on a discretization, rather than a certificate an independent party can re-check. Interval analysis offers guarantees: Jaulin computes the number of path-connected components of a set defined by nonlinear inequalities with outward-rounded intervals [16], which does prove disconnection; the proof, however, is the paving itself, and the demonstrations are two-dimensional. Canny's roadmap algorithm is exact and complete but not practical [4]. We sit between these: practical scenes, exact *verification*, with a compact proof object.

Why the field is moving. A June 2026 preprint on learning motion feasibility from point clouds observes that existing infeasibility certification "is limited to low-dimensional configuration spaces" and "often assume[s] simplified geometric environments" [2]. The 7-DOF certificate of §8, on a kinematically faithful iiwa7 with its faithful link silhouette, is a direct answer to the first half of that sentence; the second half—polytopal obstacles, one convex hull per certified body—is a limitation we share and state (§9).

To our knowledge, no prior work produces an infeasibility certificate for articulated-arm motion planning that an independent program can re-verify in exact arithmetic.

3 Method

3.1 Setting and notation

A serial revolute chain has joints $q = (q_1, \ldots, q_n)$ constrained to a box $P = \prod[q_i^-, q_i^+]$ with $q_i^\pm \in (q_i^* - \pi, q_i^* + \pi)$. Certain joints may be *locked* at a fixed angle whose cosine and sine are exact rationals with $\cos^2 + \sin^2 = 1$ ($0, \pm\pi/2, \pi$, or Pythagorean angles); locked joints carry no variable. Obstacles are static polytopes $O = \{x \in \mathbb{R}^3 : Ax \leq b\}$ with rational A, b. A *certified body* B is the convex hull of finitely many rational vertices $p_1, \ldots, p_K$ fixed in the frame of one link; a certificate concerns one such body (a claim about several bodies is several certificates). A *pair* is a couple (B, O). Two configurations $s_{\text{start}}, s_{\text{goal}}$ are given; the claim to certify is that no continuous path inside P joins them without the certified body intersecting some obstacle.

3.2 Rational forward kinematics

With $s_i = \tan((q_i - q_i^*)/2)$ one has $\cos(q_i - q_i^*) = (1 - s_i^2)/(1 + s_i^2)$ and $\sin(q_i - q_i^*) = 2s_i/(1 + s_i^2)$. Composing homogeneous transforms along the chain of a link ℓ, every entry of the link's world pose is a polynomial in s divided by the common denominator

$$D_\ell(s) = \prod_{i \in \text{chain}(\ell)} (1 + s_i^2) > 0,$$

with numerators of degree at most 2 in each variable. The world position of a body-frame point p is $X(s)/D_\ell(s)$ with $X(s) = N(s)\,p$, so the world vertices of B are $v_k(s) = N_k(s)/D_\ell(s)$, all over the same positive denominator. A locked joint contributes a constant rotation matrix with rational entries and no factor to D_ℓ. Fixed inter-link rotations of a real robot description that are $\pm 90°$ or $180°$ are signed permutation matrices and fit the same form; this is how the KUKA iiwa7 of §8.4 is represented exactly.

Constant factors are what make the kinematics *exactly* rational; the verifier (§5) re-derives N and D from the declared chain rather than reading them from the certificate. The convention is the usual one and is fixed by the reference definitions of §6: joint i is a rotation about a rational unit axis fixed in the frame of the preceding link, applied after the rational translation of that link's offset; the pose of link ℓ is the right-composition of these steps along joints $0..\ell$ inclusive, and the k-th unlocked joint reads the variable s_k.

3.3 Barrier, slab, and the disconnection lemma

A *barrier* is a rational polynomial $\varphi(s)$ of degree at most 2 per variable together with a rational $\delta > 0$. The *slab* is $\Sigma = \{s \in P : |\varphi(s)| \le \delta\}$.

Lemma 1 (slab separation)**.** *If $\{s \in P : |\varphi(s)| < \delta\} \subseteq \mathscr{C}_{\text{obs}}$, $\varphi(s_{\text{start}}) < -\delta$ and $\varphi(s_{\text{goal}}) > \delta$, then no continuous path in $P \cap \mathscr{C}_{\text{free}}$ joins s_{start} to s_{goal}.*

Proof. Along any continuous path γ from s_{start} to s_{goal}, $\varphi \circ \gamma$ is continuous and passes from below $-\delta$ to above δ, hence takes the value 0 at some $\gamma(\tau)$, which therefore lies in the open slab $\subseteq \mathscr{C}_{\text{obs}}$. □

The whole method is a way of proving the premise $\{|\varphi| < \delta\} \subseteq \mathscr{C}_{\text{obs}}$ by a finite, exactly checkable argument. In the certificates we ship, φ is supplied with the scene (in all thirteen, a single joint coordinate, $\varphi = s_1$ or s_2, chosen by the scene author, whose slab is a band of that joint); an automatic fitting pipeline (SVM on free samples labelled by side, then least squares to degree ≤ 2 per variable, rationalized) exists and reproduces the planar results, but it is not part of the trust chain and not needed for the results below.

3.4 Cell certificates by Bernstein linear programming

Fix a pair (B, O) and a box cell $C \subseteq P$. A *witness* is a point $x(s) = \sum_k \lambda_k(s)\, v_k(s)$ with polynomial coefficients $\lambda_k(s)$—constant, affine, or quadratic in s—satisfying $\sum_k \lambda_k \equiv 1$ as a polynomial identity and $\lambda_k(s) \ge 0$ on C. Then $x(s) \in B(s)$ for every $s \in C$. For each face j of O define

$$g_j(s) = b_j D_\ell(s) - a_j^\top \sum_k \lambda_k(s) N_k(s), \qquad T(s) = \delta^2 - \varphi(s)^2.$$

Since $D_\ell > 0$, $g_j(s) \ge 0$ is equivalent to $a_j^\top x(s) \le b_j$, and $T(s) \ge 0$ exactly on the slab. If, for each face, there is a constant $\mu_j \ge 0$ with

$$g_j(s) - \mu_j T(s) \;\ge\; 0 \quad \text{for all } s \in C, \tag{1}$$

then on $C \cap \Sigma$ every $g_j(s) \ge \mu_j T(s) \ge 0$, so $x(s) \in B(s) \cap O$ and $C \cap \Sigma \subseteq \mathscr{C}_{\text{obs}}$. This is a one-multiplier Putinar-type product [33] in which the sign is essential: $g_j + \mu_j T$ would certify nothing, and the test suite contains an instance where that sign "certifies" a cell that contains free configurations.

Non-negativity of a polynomial on a box is established through its Bernstein coefficients [10]: if all coefficients of the tensor-product Bernstein expansion of a polynomial on C are non-negative, the polynomial is non-negative on C. All polynomials in (1) and in $\lambda_k \ge 0$ are expanded at a common per-variable degree $d = \max(\deg\lambda + \deg N,\ 2\deg\varphi)$, where $\deg N = 2$ is fixed by the half-angle substitution and $\deg\varphi$ is the degree *declared* with the barrier (not its effective degree—a linear φ declared at degree 2 still yields $d = 4$). Every certificate reported here has $d = 4$ (affine λ, barrier declared quadratic), and the verifier recomputes the same d from the certificate's own fields. Every Bernstein coefficient is *linear* in the unknowns (the coefficients of the λ_k, the μ_j). The cell problem is therefore the linear program

$$\begin{aligned} \max\ t \quad \text{s.t.} \quad & \text{Bern}_C(g_j - \mu_j T) \ge t \ \ \forall j, \qquad \text{Bern}_C(\lambda_k) \ge 0 \ \ \forall k, \\ & \textstyle\sum_k \lambda_k \equiv 1, \qquad \mu_j \ge 0, \end{aligned} \tag{2}$$

solved with HiGHS [15]. A cell is a *collision leaf* when (2) returns a strictly positive margin (the shipped tolerance is $t^* > 10^{-6}$). That decision is taken on the floating-point LP; the multipliers are only then made exact (§3.7) and submitted to the exact verifier, which is the sole arbiter of the final verdict. No semidefinite program appears anywhere.

3.5 Outside leaves and the partition

A cell entirely outside the slab needs no witness: it is an *outside leaf* if $\text{Bern}_C(\varphi - \delta) \ge 0$ or $\text{Bern}_C(-\varphi - \delta) \ge 0$. Starting from P, the generator tests each cell—outside first, then (2) for each pair—and otherwise bisects it at the midpoint of one axis: an axis whose split pushes a whole child out of the slab if one exists, else—in the `margin` mode used by every scene reported here—the axis with the best one-step look-ahead margin (the library's own default fallback is simply the widest active axis). The axis rule affects cost only, never soundness: any midpoint partition is re-proved by the verifier from the leaf list alone. Cells are only ever bisected at midpoints; this is a contract with the verifier, which re-proves the tiling from that rule alone (§5). The search stops when every cell is a leaf (PROOF), or when a budget of leaves, time or depth is

exhausted, in which case the verdict is UNDECIDED and the open cells are reported. UNDECIDED is never read as "feasible": the method is sound, not complete.

Leaves may be certified against different pairs; a disconnection in which a body is trapped by one obstacle in part of the slab and by another elsewhere is certified by a relay of pairs across leaves (Fig. 2).

3.6 Active dimensions

For a pair whose body sits on link ℓ, the joints downstream of ℓ do not enter N_k or D_ℓ. After exactly dividing N_k and D_ℓ by every factor $(1+s_i^2)$ they share (which removes joints that are in the chain but leave the body's geometry unchanged, such as a roll about the body's own axis), the variables absent from φ, from every N_k and from D_ℓ are the pair's *passive* dimensions; the others are *active*. Along a passive axis every polynomial in (2) is constant, so the LP is built in the k active variables only, with $(d+1)^k$ Bernstein coefficients per constraint instead of $(d+1)^n$. The partition never bisects a passive axis, since no test depends on it. This is a cost lever, not a restriction of the theorem: a leaf certified in k active variables certifies the full n-dimensional cell, because the polynomials are literally constant along the other axes. Pairs that differ in their body have different passive sets; each leaf's LP is reduced to *its* pair's active dimensions, while the branching axis is chosen among the union. With one certified body per certificate, as in every certificate we ship, all pairs share the same passive set; the per-pair machinery is kept for multi-body certificates.

3.7 Exact rounding and export

The LP returns floating-point λ, μ. They are made exact as follows. The coefficients of each λ_k are blended toward the uniform barycentre, $\lambda \leftarrow (1-\alpha)\lambda + \alpha/K$ with α a small fraction of the measured face margin, so that structural zeros of $\mathrm{Bern}(\lambda_k)$—the witness typically sits on a body vertex—become strictly positive; all but one vertex's coefficients are rounded to rationals with a bounded denominator, and the last is set by subtraction so that $\sum_k \lambda_k \equiv 1$ holds exactly. The exact verifier is run on the result; if it rejects, the denominator bound is raised and the leaf is re-rounded. In every certificate reported here this loop accepted at the first denominator bound (10^6); the escalation path has never been taken on a shipped scene.

Exported leaves carry (cell, pair, λ, μ) in full dimension. When the pair's passive dimensions are absent from the *original* tensors and no common factor was divided out, the reduced witness is embedded as-is with zero exponents on the passive axes—the same polynomial, in n variables; the verifier accepts it directly. Otherwise the leaf's LP is re-solved in full dimension. Each embedded leaf is audited before export with the verifier's own leaf check, so that an incorrect embedding cannot leave the generator: it would fall back to the full re-solve and be counted. The count of *rejected* embeddings is zero on every certificate we ship; the fallback itself is not dead code—it carries all four leaves of the 4-DOF anchor scene (§8.2),

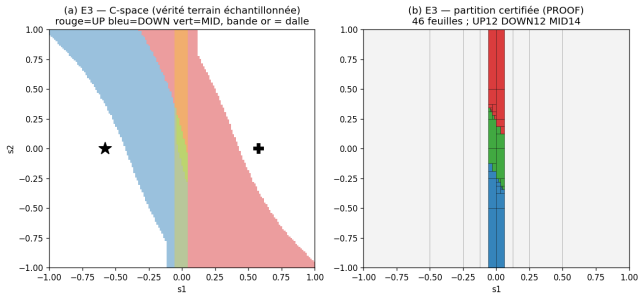


Figure 2: A planar relay certificate (2-DOF regression scene, 46 leaves). Left: sampled configuration space—the three obstacles (teeth) in red, blue and green, the slab in gold, start ($\star$) and goal ($+$) in separate free components. Right: the certified partition, zoomed on the slab; each collision leaf is coloured by the tooth whose witness certifies it (12 / 12 / 14 leaves), the middle link being trapped by three teeth in turn; the part of each leaf outside the slab is hatched—the theorem asserts nothing there—and the slab boundary $|\varphi| = \delta$ is drawn.

whose coaxial roll makes the embedding condition genuinely false. Whichever path produced a leaf, the certificate has the same format and the verifier cannot tell.

The branch-and-bound runs on a work queue of cells over several processes; the tree it explores is the deterministic tree of the serial algorithm, so parallel and serial runs produce byte-identical certificates.

4 Theorem and scope

Theorem 1 (certified disconnection)**.** *Let a certificate declare a joint box P, a serial revolute chain with rational unit axes, rational offsets and locked joints with rational* $(\cos, \sin)$, $\cos^2 + \sin^2 = 1$*; rational polytopal obstacles; a certified body as a rational vertex set on a named link; a barrier (φ, δ) with $\delta > 0$; two rational configurations $s_{\mathrm{start}}, s_{\mathrm{goal}}$; and a finite set of leaves, each a sub-box of P labelled* outside *or* collision *with a pair and exact rational (λ, μ). Suppose*

(i) $\varphi(s_{\mathrm{start}}) < -\delta$ *and* $\varphi(s_{\mathrm{goal}}) > \delta$*;*

(ii) the leaves are exactly the cells of a tiling of P obtained by repeated midpoint bisections, with no gap and no overlap;

(iii) every outside leaf C satisfies $\mathrm{Bern}_C(\varphi - \delta) \geq 0$ *or* $\mathrm{Bern}_C(-\varphi - \delta) \geq 0$*;*

(iv) every collision leaf C with pair (B, O) satisfies $\sum_k \lambda_k \equiv 1$, $\mathrm{Bern}_C(\lambda_k) \geq 0$ *for all k,* $\mu_j \geq 0$ *and* $\mathrm{Bern}_C(g_j - \mu_j T) \geq 0$ *for every face j of O, where g_j, T are formed from the declared chain, body, obstacle and barrier as in §3.4.*

Then no continuous path in P from s_{start} to s_{goal} keeps the certified body out of every obstacle.

Proof. Let γ be such a path. By (i), $\varphi(\gamma(0)) < -\delta < 0 < \delta < \varphi(\gamma(1))$, so by the intermediate value theorem applied to the continuous map $\varphi \circ \gamma$ there is τ with $\varphi(s^*) = 0$, where $s^* = \gamma(\tau)$. By (ii) s^* lies in exactly one leaf C. C cannot be an outside leaf: (iii) would give $\varphi(s^*) \geq \delta$ or $\varphi(s^*) \leq -\delta$, both impossible since $\delta > 0$. Hence C is a collision leaf with

pair (B, O). Since $T(s^*) = \delta^2 - \varphi(s^*)^2 = \delta^2 > 0$, (iv) gives, for every face j, $g_j(s^*) \geq \mu_j T(s^*) \geq 0$, which after division by $D_\ell(s^*) > 0$ reads $a_j^\top x(s^*) \leq b_j$; and $\sum_k \lambda_k(s^*) = 1$ with $\lambda_k(s^*) \geq 0$ puts $x(s^*)$ in $B(s^*)$. So $x(s^*) \in B(s^*) \cap O$ and $\gamma(\tau)$ is a collision configuration, contradicting the hypothesis. □

What the certificate establishes. Every $s \in P$ with $|\varphi(s)| < \delta$ lies in a leaf; it cannot be an outside leaf, hence it is a collision leaf, hence it is in $\mathscr{C}_{\text{obs}}$. The certificate therefore establishes $\{s \in P : |\varphi(s)| < \delta\} \subseteq \mathscr{C}_{\text{obs}}$—the *open* slab—and Lemma 1 needs no more. Nothing is claimed about the boundary $\{|\varphi| = \delta\}$, and nothing needs to be. The outside test in (iii) is non-strict, and deliberately so: the proof selects a point at which φ vanishes, so the slab boundary never has to be adjudicated; a strict test would reject certificates the non-strict one accepts and prove nothing more. Condition (iii) is exactly the shipped test.

Three kinds of hypotheses. The mechanised proof of §6.3 makes a distinction the statement above blurs. Conditions (i)–(iv) and $\delta > 0$ are what the proof uses. The requirements on the chain—rational unit axes, $\cos^2 + \sin^2 = 1$, $q^* = 0$—are not: they are what makes the formal object a rigid-body kinematics rather than an arbitrary rational map, i.e. conditions of *meaning*. So is the requirement, not tested by either verifier, that s_{start} and s_{goal} lie in P: a start outside the box makes the theorem vacuously true, and the scene cross-check is where that belongs. Finally, that cuts are at midpoints, that λ_k has the declared degree and that there is one μ_j per face are *agreement* conventions that let the verifier reconstruct the generator's tiling; no proof depends on them.

Remark (sides, not points). Start and goal enter the proof only through the intermediate value theorem. Any collision-free path in P avoids the open slab, so φ keeps a constant sign along it: the certificate separates every configuration of P with $\varphi < 0$ from every configuration with $\varphi > 0$, not merely the two it names. We state this as a remark because the formalised statements of §6 are the point-to-point ones; the proof is the same with (i) replaced by $\varphi(s_0) < 0 < \varphi(s_1)$. In practice it means a certificate answers every start–goal query on opposite sides of its barrier at the cost of two polynomial evaluations.

Joint angles and s. The theorem and its formalisation are statements in s-space. Since $s_i = \tan((q_i - q_i^*)/2)$ is a homeomorphism from $(q_i^* - \pi, q_i^* + \pi)$ onto $\mathbb{R}$, a continuous path in joint angles inside the declared box is a continuous path in s and conversely, so the statement transfers to joint space; this translation is elementary and is not formalised.

Three remarks. First, the theorem quantifies over the *whole* box P, passive dimensions included: passivity means the tiling never needed to split those axes, not that they were excluded from the claim. Second, soundness rests only on (i)–(iv), each of which is decidable in exact rational arithmetic from the certificate's own data; nothing about how the certificate was found enters the proof. Third, nothing is claimed when the generator returns UNDECIDED.

Restrictions. Serial revolute chains (prismatic joints are outside the present scope); $q^* = 0$ for free joints, or locked joints with exactly rational $(\cos, \sin)$; the box P lies within $(q_i^* - \pi, q_i^* + \pi)$ per joint, so there is no wrap-around and the theorem is about the declared box, which may be smaller than the factory limits (every figure and interactive artefact states the box in degrees; the iiwa7 scenes of §8 declare $\pm 70°$ on the three active joints and $\pm 143°$ on the four passive ones, against $\pm 170°$ and $\pm 120°$ limits in the robot's Drake model, and say nothing beyond that box); static obstacles in H-representation with rational data; convex certified bodies with rational vertices; rational s_{start} and s_{goal} (a demonstration angle with irrational $\tan(\theta/2)$ is replaced by a nearby rational configuration, and the strict inequality (i) is checked there). The barrier is part of the certificate's statement, not of its proof: the theorem is "given this φ, the slab lies in $\mathscr{C}_{\text{obs}}$ and separates", and the consumer may inspect φ.

Kinematic fidelity. Our robot models are exact as internal objects. Their fidelity to a physical robot is bounded by the published description: the joint axes of the iiwa7 SDF distributed with Drake are quaternion-rounded and aligned with the coordinate axes only to about $3.7 \cdot 10^{-6}$ rad. We certify a rational kinematics whose forward kinematics agrees with Drake's on 1 500 random configurations to $1.99 \cdot 10^{-6}$ m at the flange (§8.4)—faithful to the published URDF at that level, not "the exact iiwa" in an absolute sense, because no such description exists. The same holds for link geometry: the certified body of §8.4 is a 40-vertex convex hull of the visual mesh, rationalized, agreeing with Drake's link to $1.67 \cdot 10^{-6}$ m over 200 random configurations and all 40 support vertices ($1.68 \cdot 10^{-6}$ m over 400).

What is proven about what. The verifier establishes Theorem 1 about the robot and obstacles *declared in the certificate*. A separate cross-check program compares that declaration, field by field, with the scene file the user authored; a certificate whose declared geometry has been perturbed is internally valid and is rejected by the cross-check, which is the intended division of labour. Tying the scene file to a physical cell remains a human responsibility, as with any model-based safety argument.

5 The independent verifier

The verifier is the reason to prefer this certificate over a floating-point one, so we describe it in some detail.

Size and dependencies. `verify.py` is 499 lines of Python. It imports `json`, `fractions` and `math.comb`—three standard-library names, and nothing else; in particular it imports nothing from the generator, a property checked in the test suite by

walking the module's abstract syntax tree. Every number it manipulates is a `Fraction`. It has no solver, no floating point, and no notion of Bernstein bounds beyond a self-contained 53-line block (38 lines of code) implementing the exact tensor-product Bernstein transform.

What it recomputes rather than reads. From the declared chain it re-implements the half-angle substitution and composes exact homogeneous transforms, so N_k and D_ℓ are its own; a certificate cannot supply convenient numerators. From λ and μ it *forms* $g_j - \mu_j T$ itself; the certificate carries only λ and μ, so the unsound sign of §3.4 cannot be smuggled in. From the leaf list alone it re-proves condition (ii): since the generator only ever bisects at midpoints, the tiling is reconstructed by recursion—a cell is either a leaf or splits at its midpoint along some axis into two cells that must each be covered—and both a missing leaf (a hole) and a duplicated leaf (an overlap) are detected. The Bernstein expansions are computed at the same degree the generator used, which is part of the generator–verifier contract; a lower degree would yield looser bounds and false rejections (never false acceptances).

The five checks , in order: (0) hypotheses—rational unit axes, $\cos^2 + \sin^2 = 1$ for locked joints, $\delta > 0$, box within the no-wrap-around range; (i) the strict side conditions on s_{start} and s_{goal}; (ii) the exact tiling; (iii) every outside leaf; (iv) every collision leaf. On any failure it returns `(False, reason)` rather than raising: malformed inputs are caught as `KeyError`, `ValueError`, `ZeroDivisionError` or `TypeError` and reported as a rejection. This is a convenience for callers, not a hardening claim—the verifier is not a fuzz target, and a sufficiently ill-formed input can still abort it. Soundness does not depend on it: an aborted verification is not an accepted certificate.

Adversarial hardening. The test suite mutates valid certificates and requires rejection: negative or oversized μ; $\sum \lambda \neq 1$; scaled or negative λ; a shifted cut; a removed leaf (hole); a duplicated leaf (overlap); $q^* \neq 0$; δ scaled by 20; $\delta < 0$; s_{start} or s_{goal} on the wrong side; the sign of φ; obstacle A or b; a hull vertex; a link length; the body's link index; a collision leaf relabelled outside and vice versa; permuted obstacles; corrupted theorem and substitution fields; a shrunk box; a cell outside the box—26 mutations on a planar certificate, plus a spatial suite (non-unit axis; wrong axis; offset that moves the body out of the trapping band) and 6 mutations on a locked-joint certificate. The locked-joint suite is the most instructive: a certificate whose locked pitch is moved to another exactly rational angle must be rejected, but only if the proof *depends* on that angle. On a scene where the witness sits on a proximal point unaffected by the lock, moving it is not detected—and that is correct, the proof does not use it. We therefore constructed a scene in which the lock displaces the trapped wrist out of a narrow wall, and there the mutation is rejected. Testing a verifier means testing the mechanism, not ticking a box.

Two layers. As noted in §4, `verify.verify` checks internal validity; `scene_matches_cert` ties the declaration to the authored scene. Both are run by the command-line `verify`. Keeping them separate keeps the sacred module small and its contract clean.

Reuse as a guard. Since the export contract of §3.7 the generator calls the verifier's own collision-leaf check on every embedded leaf before writing the certificate. We chose this over a re-implementation on purpose: a guard that *is* the arbiter cannot drift from it. The price is that the verifier runs twice per certificate—once per leaf as a guard, once on the whole file—about 4.9 s on the flagship. We consider that cheap.

Cost. Verification of the 7-DOF flagship certificate (two collision leaves, 40-vertex body, 7 free joints) takes 2.4 s; the 4-DOF anchor scene of §8.2 takes 0.04 s. Verification time grows with the number of leaves and with $(d+1)^n$, since the verifier expands in full dimension; it has never been the bottleneck.

A second verifier. Everything in this section is about a program one reads. §6 describes a second implementation of the same five checks in Lean 4, proved sound, and the measured agreement of the two.

6 A verifier proved sound in Lean

The Python verifier of §5 is small enough to read; this section replaces reading with a proof. We re-implemented the verifier in Lean 4 [9] and proved, with Mathlib [36], that whenever it accepts a certificate, no collision-free path exists. The Python program remains the one a user runs; the Lean development says what accepting a certificate *means*, and the two are compared on every certificate we ship. Every number below is read from two archived runs on a quiet machine (10 October 2026, Apple M4, 10 cores, 24 GiB; each run started only once the 1-minute load average was below 1.5), and the development is replayed by continuous integration on a second architecture.

6.1 What was built

The executable core—tensor arithmetic, the Bernstein transform, the rational kinematics, the certificate type, the five checks and a `main`—is 1 087 lines of Lean in six files and compiles to a binary, `cnp-verify`, that reads a certificate and prints `PROOF` or a rejection with its reason, exactly as the Python verifier does. On top of it sits a proof layer of 2 501 lines and 145 theorems, among them the 19 proof obligations of the plan: the tensor operations compute the polynomial operations they claim; non-negative Bernstein coefficients imply non-negativity on the box; an accepted tiling covers the box; an accepted outside leaf has $|\varphi| \geq \delta$ on its cell; an accepted collision leaf is in collision wherever $|\varphi| \leq \delta$ on its cell; the common denominator is positive; the polynomial kinematics

N/D is the reference kinematics; $s = \tan(q/2)$ gives the rotation of angle q; and Rodrigues' matrix is a rotation for a unit axis. There is no `sorry`. An audit script asserts that every statement depends on the three standard axioms of Lean—`propext`, `Classical.choice`, `Quot.sound`—and on nothing else.

6.2 The two statements

Two theorems are proved about the checker `check`. The first, `soundness`, is stated with the checker's own polynomial kinematics:

```
theorem soundness (c :  Certificate) :
check c = true →
¬∃ γ :  C(unitInterval, Fin c.n → ℝ),
(∀ t, γ t ∈ c.box.toSet) ∧ γ 0 = vecR
c.sStart ∧ γ 1 = vecR c.sGoal
∧ ∀ t, CollisionFree c (γ t)
```

where `CollisionFree` says that, for every declared obstacle, the convex hull of the body's vertices moved by N/D is disjoint from the closed polytope. The second, `soundness_ref`, is the same statement with `CollisionFree` replaced by a version written against *reference* definitions that contain no polynomial and never mention the checker: a joint is Rodrigues' rotation $I + \sin\theta\,[a]_\times + (1-\cos\theta)[a]_\times^2$ at $(\cos\theta, \sin\theta) = \big((1-s^2)/(1+s^2),\, 2s/(1+s^2)\big)$ for an unlocked joint and at its rational $(\cos, \sin)$ for a locked one; the pose of link ℓ is the right-composition of (translate by the offset, then rotate) along joints $0..\ell$, the k-th unlocked joint reading s_k; the body at s is the convex hull of its vertices under that pose; obstacles are closed half-space intersections in the world frame. The statement reads eight fields of the certificate—n, the chain, the body's link, its vertices, the obstacles, the box, s_{start} and s_{goal}. The barrier, δ, the leaves and their multipliers are the material of the proof and do not occur in the conclusion.

The reference definitions were re-read, independently of their author, against both the Python verifier and the textbook conventions—right-handed cross product, Rodrigues' formula, half-angle identities, affine composition on the right, joints $0..\ell$ inclusive—since a proof shows that the checker agrees with the definitions, not that the definitions say what a roboticist means. Nothing had to be changed. Three things the statement does not say: it is a statement in s-space (the translation of a joint-angle path within $(-\pi, \pi)$ into an s-path, a per-coordinate homeomorphism, is not formalised); it is about one body and the listed obstacles, and touching counts as a collision; it is soundness only—nothing says the checker accepts every true disconnection, nor anything about the program that produced the certificate.

6.3 What soundness depends on

A mechanised proof also says which tests of the checker the theorem *needs*, and the answer sorts the verifier's conditions into three kinds. *Soundness* hypotheses are the ones the proof uses: $\delta > 0$, the strict side conditions (i), coverage of the box by the leaves, and the two leaf tests (iii) and (iv) with the positivity of the common denominator. *Meaning* hypotheses are tests the proof never reads but which make the formal object the robot's configuration space: $q^* = 0$ (which is what makes $s = \tan(q/2)$ the right reading of the certificate), unit axes and $\cos^2 + \sin^2 = 1$ (without them `soundness_ref` would still be true, about maps that are no longer rigid motions), and the position of s_{start} and s_{goal} inside the box (not tested by either verifier; a start outside P makes the theorem vacuously true). *Agreement* conventions are tests that let the verifier reconstruct the generator's tiling but that no proof uses: that cuts are at midpoints, the degree test on λ_k, the count of the μ_j—a missing μ_j is read as 0, which only makes the condition stricter. In particular `soundness` does not use the kinematic check at all, and `soundness_ref` uses one fact of it: that n is the number of unlocked joints. We found this sorting more informative than the proof itself, and it is reflected in the wording of §4.

6.4 From the theorem to a binary, and for one scene to a theorem

A `PROOF` printed by the binary rests on the theorem plus what the theorem is not about: the JSON parser (ordinary unverified code; `cnp-verify -dump` prints the object actually checked), the Lean compiler and its run time (rationals on GMP, arrays, the `extern` replacements of core functions), the few lines of `main`, and the kernel and Mathlib themselves. For one shipped scene these items drop out. The hypothesis `check wallK3 = true` for the 3-joint sealed wall was proved by evaluation *in the kernel*, so the disconnection of that scene is a theorem of Lean/Mathlib with the three standard axioms: no compiler, no binary, no parser—the certificate is a term written out in a Lean file, and that the term is the content of the JSON file is checked at run time by two independent readers, with a negative control. The kernel evaluation had to be cut into 34 stages glued by lemmas, each stage a kernel evaluation with its own memory; the whole file takes 102.8 s (101.6 s in the kernel) with a peak of 5.63 GiB. The plain `decide` tactic cannot do it at any resource limit (core rational arithmetic is irreducible to the elaborator), a single kernel evaluation exceeds 9 GiB on this smallest certificate, and the staged proof does not carry to the 4-joint anchor or to the 7-DOF flagship: three kinds of stage each exceed 9 GiB there, because the array operations the checker is written with are quadratic in the kernel ($\times 25$ to $\times 30$ from 125 to 625 entries, against $\times 5$ for a loop). Carrying the flagship to that status needs a twin of the checker written for the kernel and proved equal to `check`; we have not attempted it. Lean's `native_decide` would prove all three in seconds, but it adds the compiler to the trust base as an explicit axiom per use; we keep such statements in a separate file, named as such, and the audit refuses the axiom anywhere else.

Table 1: The two verifiers on the 13 shipped certificates, same quiet-machine run (archived `lean_parity.json`, 10 October 2026). Python: `verify.py` in process; Lean: the time `cnp-verify` reports, process start-up excluded. All `PROOF` on both sides.

Certificate	Python	Lean	Lean/Python
Planar relay (S1)	0.087 s	0.095 s	×1.09
Comb (S2)	0.171 s	0.126 s	×0.73
Shoulder–elbow (S3)	0.040 s	0.033 s	×0.82
iiwa-like bin (S4)	0.132 s	0.081 s	×0.61
iiwa7 shelf, earlier model (S5)	3.004 s	1.655 s	×0.55
iiwa7 shelf, flagship (S6)	2.047 s	1.669 s	×0.82
iiwa7 stacked crate	1.876 s	1.594 s	×0.85
iiwa7 safety guard	1.978 s	1.589 s	×0.80
Sealed wall $k = 3$	0.011 s	0.009 s	×0.85
Sealed wall $k = 4$	0.052 s	0.050 s	×0.97
Sealed wall $k = 5$	0.205 s	0.215 s	×1.05
Sealed wall $k = 6$	1.068 s	0.968 s	×0.91
Sealed wall $k = 7$	6.031 s	5.863 s	×0.97
Total	16.702 s	13.947 s	×0.83

6.5 Agreement with the Python verifier, measured

Agreement between the two verifiers is measured, not proved. On the 13 shipped certificates both print `PROOF`; on an adversarial corpus of 48 certificates (43 to reject, 5 to accept) they return the same verdict and the same reason in 48 cases; on 1 000 seeded differential fuzz cases, 498 are `PROOF` on both sides, 487 rejected on both, 15 raise in Python and are rejected in Lean, and there is no verdict divergence. Their forward kinematics agree on 1 299 exact rational values. Table 1 gives the time of each checker on the same run; the Lean binary is somewhat faster on every scene but one, which was never the point. Continuous integration rebuilds the development, re-runs the axiom audit, re-checks the staged kernel proof (200 s on a 4-core Linux runner) and replays the parity suite on every push: 10 min 8 s in all.

6.6 What this changes

Before, the trust anchor was a 499-line program a person reads in an afternoon. Now the reading is of a statement and of a dozen reference definitions that contain no polynomial, and the kernel checks the rest; for one scene the kernel also checks the certificate. The Python verifier is kept because it is what a user without a Lean toolchain runs, and because two independent checkers that agree on 1 061 inputs are better evidence than either alone. What the formalisation does not cover is listed above so that nobody over-reads it: the $q \leftrightarrow s$ translation, the parser and compiler behind a binary `PROOF`, the fidelity of a certificate to a scene or a robot, and anything about the generator.

7 Cost model and the scope of low-cost certifiability

7.1 A selection effect, stated as scope

Every disconnection we certify at low cost has the same shape: a body early in the kinematic chain is trapped, and no setting of the joints downstream of it frees it. We call such disconnections *proximal*. This is not a property of disconnections in general—a whole arm threading a narrow window is a disconnection in which the collision depends on every joint—but a selection effect of the method: a low-degree scalar barrier and a slab certify cheaply exactly those separations that a few joints already realize. It happens to be the class that matters in practice (bins, shelves, guards), and we state it as the scope of the low-cost regime rather than leave it to be discovered.

The effect was measured, not assumed, on the real robot. On the iiwa7 link 3 we swept the three active joints × six directions × two obstacle motifs (a half-space "ceiling" and a plate "wall to cross") for a robust separation margin. Exactly one proximal trap is robust: shoulder pitch against an overhead obstacle (+175 mm at the design stage). Base yaw and arm roll are non-positive in every direction (from exactly 0 mm along $-z$ down to -299 mm); crossing a vertical wall is negative everywhere. The compact link is anchored at the shoulder: it rotates in place without displacing, and it can never lie entirely on one side of a vertical plane. Our three use-case scenes (§8.5) therefore share one mechanism and differ in obstacle geometry, joint limits, slab, poses and claim; we say so in each use-case scene file. A design lesson came with it: the faithful compact silhouette of the real link removed the lever arm that had made segment-shaped bodies easy to trap by base yaw (a 6 mm base-yaw separation was measured and refused as fragile). On a realistic arm, the robust separator is a joint with real lever on the body—here the shoulder pitch—or an enclosing obstacle, not the base yaw.

7.2 Passive dimensions are free; active dimensions cost $(d+1)^k$

Two quantities govern cost: the number of leaves in the partition and the size of each leaf's LP. Measured on the 5-DOF iiwa-like bin (§8.3) with the number of *passive* joints varied from 0 to 3 while the three active joints are held fixed, the leaf count is constant at 8 and the reduced LP is constant at 766 rows; the full-dimension LP that an unreduced solver would build grows $766 \to 3\,782 \to 18\,814 \to 93\,878$ rows, a factor 4.9 per passive joint, i.e. $(d+1)$ with $d = 4$. The active-dimension reduction is thus 122.6× at six joints on that family, and 443× on the 7-DOF flagship (1 070 rows reduced against 473 870 full). On a two-body variant the reduction is *per pair*: the proximal pair pays its own three active dimensions (766 rows) where a global reduction over the union of active sets would charge it 18 814—a factor 24.6. Passivity is not a restriction of the theorem (§4); it is a lever.

Active dimensions are the cost. On a family of synthetic disconnections sealed by Bernstein bounds—the obstacle is

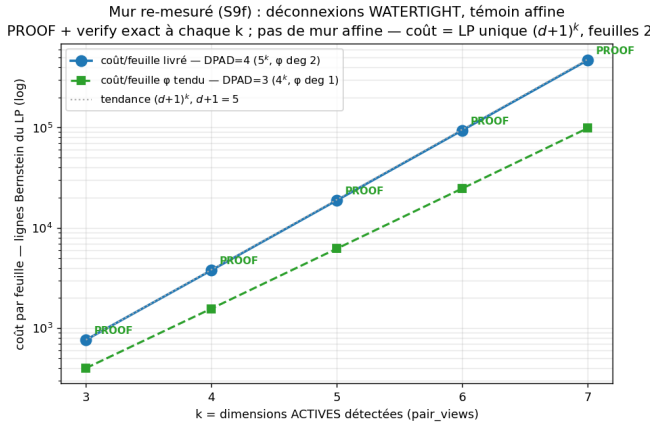


Figure 3: Cost in active dimensions on the sealed synthetic family: per-leaf LP rows (log scale) against the number k of active joints. PROOF with exact verification at $k = 3 \ldots 7$ (766 to 469 006 rows, a geometric-mean factor of 4.97 per added dimension, i.e. $(d+1) = 5$ with $d = 4$). The $k = 8$ point (about 2.3 million rows) is marked UNDECIDED: a single LP outlasts the 300 s budget.

proven to cover the body's reach on the band, so each scene is a genuine disconnection—with k active joints and a linear barrier, the affine witness certifies every k from 3 to 7 with 2–4 leaves; the per-leaf full-dimension LP runs 766, 3 782, 18 814, 93 878, 469 006 rows (a factor 4.97 per added active dimension, geometric mean), and at $k = 8$ (about 2.3 million rows) a single LP outlasts the 300 s deadline—it returns after 477 s—and the verdict is UNDECIDED on budget (Fig. 3). The practical frontier of this family is therefore the size of one LP, not certifiability—and it is a frontier that column generation or a lazily-materialized Bernstein basis could push, whereas a wall in leaf count could not. We are careful about what this family shows: barriers are simple and leaves are few. Disconnections that need many leaves *and* many active dimensions (cages, windows) sit in the product leaves $\times(d+1)^k$ and were not measured; that is the open regime.

An earlier run of the same family had suggested a wall at $k = 5$. It was an artefact: a silent depth cap in the branch-and-bound, and a scene that was not actually disconnected (the second episode of §8.7). We report it because the diagnosis changed how the engine reports termination—the serial engine, the default, now records why it stopped (certified, depth exhausted, leaf budget or time budget)—and because it is the kind of measurement error a certificate should protect one from.

7.3 Where the time goes

After the export contract of §3.7, certifying the 7-DOF flagship takes 46.7 s on a laptop CPU: 39.1 s building the symbolic forward kinematics, 1.3 s for the branch-and-bound proof, 6.3 s for export—of which 4.9 s is the exact verifier running twice, once per leaf as a guard and once on the whole certificate. The proof itself is a second; the trust chain is five; the rest is symbolic setup that a cache or a faster kinematics backend would remove. Before the export contract, certification took 726 s, of which about 580 s was re-solving the two leaves' LPs in full dimension (291 s per leaf), and the cost was not in rows: at nearly equal row count (473 870 vs 469 006), the 40-vertex faithful hull multiplies LP columns by 14.2 (327 vs 23) and export time per leaf by 39.9 (about columns$^{1.39}$). Each hull vertex carries its own multiplier, so a faithful silhouette multiplies columns in the reduced LP too—but the reduced LP stays small (1 070 rows $\times$ 167 columns on the flagship), which is why the reduced witness, embedded, is the right thing to export.

7.4 The nature of the certificate

Table 2 is about kind, not speed. The scenes, hardware and proof objects differ, and the disconnections we certify exploit passivity theirs may not.

8 Experiments

All runs: MacBook (Apple Silicon), CPU only, Python 3.12, HiGHS via `highspy`. Every benchmark directory records its commit hash (and, since June 2026, whether the working tree was clean; the three iiwa7 benchmarks quoted below were run from a tree whose certificates were not yet committed, and say so), and every sampling step is seeded. Ground truth is always established *before* certification (start and goal free; no free sample in the slab; free space on both sides) with a collision oracle independent of the certificate: point sampling along the link for segment-shaped bodies, and a convex-body-versus-polytope LP feasibility test for the faithful iiwa7 link. Since the leaky-scene episode of §8.7, ground truth also includes a corner-biased pass. Verification times are for the exact verifier of §5.

8.1 Planar regression

Two planar 2-DOF scenes fix the ground: a three-obstacle relay in which the middle link is trapped by three "teeth" in turn (46 leaves: 38 collision across three pairs, 8 outside; Fig. 2) and a scene with a learned barrier (78 leaves). The relay reproduces, cell by cell, the certified margin of an earlier two-dimensional Bernstein-LP implementation to 10^{-7}; the learned-barrier scene reproduces its partition. A 3-DOF "comb" lifts the relay by one passive joint and was the first scene on which the widest-axis rule wasted its depth on the passive joint (736 leaves, 256 of them failed, UNDECIDED) while a one-step look-ahead rule certified in 54 leaves; restricting axis choice to active dimensions (§3.6) later let the widest-axis rule certify it too, in 46 leaves.

8.2 A 4-DOF anchor

We reproduce the shoulder-elbow 4-DOF scenario of Li and Dantam [19]—spherical shoulder plus elbow, target blocked by a panel—as an approximate reproduction, since they publish topology and task but no geometry; all dimensions are

Table 2: Kind of certificate, not speed. Scenes, hardware and proof objects differ across rows.

	Li–Dantam [23]	Henrion et al. [14]	C-IRIS [7]	This work
Question	disconnection	disconnection	free regions	disconnection
Setting	arms, 5–6 DoF	abstract, $n \leq 3$	arms, 7 DoF	arms, ≤ 7 DoF
Certificate	triangulated manifold	moment-SOS	SOS	Bernstein-LP witnesses
Checked by	float collision checker	SDP tolerance	SDP tolerance	exact rational, independent
Hardware	RTX 4070 + i9	—	—	laptop CPU
Complete?	asymptotically	nec. & suff.	—	sufficient only

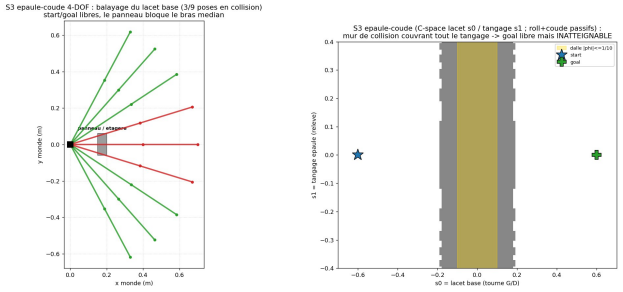


Figure 4: The 4-DOF anchor. Left: top view, sweep of the base yaw between start and goal (both free); poses in the band plant the upper arm in the panel. Right: configuration space (yaw × pitch), roll and elbow passive: the collision wall spans all admissible pitch and contains the slab; start and goal lie on either side—the goal is free but unreachable.

ours and are tabulated against theirs in the repository. The upper arm is trapped by a thin vertical panel over a band of base yaw for all admissible pitch; roll and elbow are passive (Fig. 4). PROOF in 8 leaves, engine 0.043 s, verification 0.041 s. Ground truth: 0 free in the slab over 150 000 samples; the panel shrunk in width gives UNDECIDED. Their reported time for the same class is minutes; we do not compare speeds (§7).

8.3 A 5-DOF iiwa-like bin

A 7-joint S-R-S chain with iiwa-like link lengths and the two wrist joints locked at 0 (5 DOF), reaching into a deep bin with a sealed front wall. The trapped body is the shoulder–elbow segment: the three proximal joints are active for the pair, the two unlocked distal joints are passive. Ground truth 0 free over 40 000 samples. PROOF, 8 leaves; engine 0.15 s, verification 0.16 s, 1.14 s end to end. Releasing the locks one at a time gives the passive-dimension calibration of §7.2 (8 leaves throughout; 3.3 s engine and export at 6 DOF, also before the export contract). The scene met our 5–6-DOF gate with more than three orders of magnitude of margin on time and leaf count, and it is where the proximal insight of §7 was first written down.

8.4 The KUKA iiwa7 flagship

Kinematics. From the iiwa7 SDF distributed with Drake we fold the constant inter-link transforms into 19 chain entries: 7 variable joints about the coordinate z-axis and 12 locked joints realizing the signed-permutation inter-link rotations ($\pm 90^\circ$, 180°) with $\cos, \sin \in \{0, \pm 1\}$; offsets are rationalized to denominator 10^6. The composition convention $T_i(\theta) = X_{PF,i}\,\mathrm{Rot}(z,\theta)\,X_{ML,i}$ with the constants folded into $G_i = X_{ML,i}X_{PF,i+1}$ is what makes this exact; a naive conjugation of the rotation through the joint frame is wrong because it displaces the translation. Forward kinematics agrees with Drake's on 1 500 configurations drawn uniformly in ± 2.9 rad on every joint—beyond the $\pm 120^\circ$ factory limits of joints 2, 4 and 6—to $1.99 \cdot 10^{-6}$ m (maximum position error at the flange); the SDF's own joint axes are aligned with the coordinate axes only to $3.67 \cdot 10^{-6}$ rad, so this is the fidelity of the published description itself.

Body. The certified body is link 3: the convex hull of Drake's visual mesh (1 301 hull vertices) is reduced by support sampling in 40 directions on a Fibonacci sphere to 40 true extreme points, expressed in the chain frame after joint 3 and rationalized. Against Drake's link, the hull agrees to $1.67 \cdot 10^{-6}$ m over 200 random configurations and all vertices.

Trap. Designed by measurement (§7): $\varphi = s_2$ (shoulder pitch), $\delta = 1/5$, an overhead shelf in H-representation with underside at $z = 17/25 = 0.68$ m. With the arm upright the link's top reaches 0.808 m and strikes the shelf; at the start and goal poses (shoulder pitch $\mp 61.9^\circ$) it tops out at 0.597 m and passes beneath (0.544 m at the $\pm 70^\circ$ edge of the joint box). Margins: the trapping band penetrates the shelf by 91.5 mm; start and goal clear it by 83.3 mm. Ground truth with the convex-body oracle: start and goal free; 0 free in the slab over 40 000 uniform samples and at 448 slab corners (2^6 extremes of the non-barrier joints × 7 barrier levels); all 16 combinations of extreme distal-joint values in collision (the redundancy invariance the proof will make exact); free on both sides.

Result. PROOF; exact verification OK; 2 leaves (both collision, none outside); active dimensions (q_1, q_2, q_3) measured, $(q_4, \ldots, q_7)$ passive; reduced LP 1 070 rows against 473 870 full (443×); certify 46.7 s (39.1 s symbolic FK build, 1.3 s branch-and-bound, 6.3 s export), verify 2.42 s; embedded leaves 2, re-solved 0, rejected 0; dissonance counter 0 (Fig. 1).

8.5 Three use cases, one robot

On the same iiwa7 chain and body, two further scenes with the same measured mechanism and different geometry, limits, slab and claim:

- **Bin picking / TAMP pruning.** A stacked euro crate ($600 \times 400 \times 220$ mm) whose underside at 0.720 m the upright arm strikes: the object below is unreachable until the crate above is removed. Joint box $\pm 62^\circ$, $\delta = 3/20$, measured window 138.7 mm placed to give $+66.9$ / $+71.7$ mm. PROOF, 2 leaves, verify 2.43 s. The certificate lets a task planner prune the "reach directly" branch with a proof rather than a timeout.
- **Safety guard.** A thin horizontal guard (50 mm, ± 0.60 m wide, underside 0.695 m) between the arm and an operator zone. Joint box $\pm 66^\circ$, $\delta = 9/50$, window 168.4 mm giving $+83.1$ / $+85.3$ mm. PROOF, 4 leaves (2 collision, 2 outside), verify 2.50 s. The claim a safety file may carry is precisely the theorem: within the declared joint box, with static obstacles as declared, no continuous motion brings the certified body past the guard—nothing about dynamics, sensing or configurations outside the box.

The original bin-picking storyboard ("cross the crate wall") was abandoned after the lever measurement showed it infeasible on the faithful body; the stacked-crate version is what the geometry supports.

8.6 Summary

Table 3 gathers every certified scene.

8.7 Two episodes the certifier refused

We record two occasions on which dense sampling declared a disconnection and the certifier did not, and the certifier was right.

1. **The micro-channel (planar).** A regression scene with teeth shrunk by 5%: an 11^3 grid finds no free sample in the slab; a denser sampling finds free configurations in it; the engine refuses (UNDECIDED) and never writes a certificate. The grid was fooled; the certifier was not. This is now a permanent test.
2. **The leaky scene (7-DOF synthetic, authored by us).** A wall-bench scene at five active joints whose obstacle was the bounding box of 4 000 random samples of the body's reach plus a margin. Ground truth: 0 free over 8 000 uniform samples. The certifier refused, and at higher depth and full budget still refused. Free configurations survived in the corners of the joint box—the link's tip protruded 8 to 40 mm past the box—a region of measure too small for uniform sampling to hit. We had written a scene that was not disconnected and a ground truth that said it was; the tool believed neither. The fix—sealing the obstacle by Bernstein bounds so that it provably covers the reach—is what made §7.2 possible, and corner-biased sampling became part of every ground-truth script.

A third episode belongs to the same family though it involves no certificate: two visualization artefacts (a figure that drew only the certified hull, a 3-D page whose replay moved a single joint) passed every automated fidelity check and were caught by the human validation gate. Correctness of what is drawn and sufficiency of what is drawn are different properties; the verifier guarantees the first for certificates, and only a person checked the second for figures. We return to this in §10.

8.8 Reproducibility

The verifier, the scene files and the certificates of every row of Table 3 are released with the paper, together with the dated benchmark files they are read from; `make reproduce` regenerates every row from a clean tree, one process per scene. The verifier is a single standard-library module: `cnp.verify.verify_file` re-proves a certificate in exact arithmetic with no third-party package installed (checked under an interpreter with site-packages disabled). The command-line form `cnp verify <cert> <scene>` adds the cross-check against the authored scene file, which parses YAML and needs the package's dependencies. Regenerating certificates requires the generator's dependencies (NumPy, SymPy, PyYAML, HiGHS); Drake is needed only to re-derive the frozen iiwa7 kinematics and body from Drake's model. Figure 1 reads its numbers from the dated benchmark file, not retyped.

9 Limitations and future work

Not measured. Disconnections requiring many leaves at many active dimensions—the product leaves $\times (d+1)^k$—remain the open regime (§7.2). Our sealed-wall family has few leaves; our real scenes have three active dimensions. A cage or a window would test both at once.

Geometry. One convex hull per certified body; a full mesh would need a convex decomposition and one pair per piece—the multi-pair machinery exists and the per-pair reduction applies, but the leaf count is unmeasured. Obstacles are static polytopes; dynamics, sensing and moving obstacles are out of scope, as is wrap-around (the theorem is about a declared box within one turn).

Kinematics. Serial revolute chains; prismatic joints are not handled. Fidelity to a physical robot is bounded by the published URDF (§4).

Barrier. The certificates we ship use scene-provided barriers, all linear in one joint. The automatic fitting pipeline is validated with exact verification on a planar scene; it also found the barrier of a 3-DOF spatial scene that was certified before the spatial verifier existed. On a passive joint it over-fits (a structured fit that zeroes passive coefficients is the obvious next step).

Time. The dominant cost is now symbolic kinematics construction: 39 s of 47 on the flagship, paid once per process (a second, cache-warm pass at export costs about 1 s); caching per scene, or Drake's rational kinematics directly, would remove most of it. The verifier runs twice as a guard. Single-LP size at $k \geq 8$ is the frontier of the active-dimension regime;

Table 3: All certified scenes. "Active" is the number of joint dimensions on which the trapped body's geometry depends: it governs the size of each leaf's LP, not the scope of the theorem, which covers every joint of the arm (§4); the sealed-wall rows raise the active count to 7. "Engine" is the branch-and-bound decision alone; "certify" adds certificate export (and, for the iiwa7 scenes, the 39 s symbolic kinematics build); "—" marks a phase the older harness did not time. All PROOF, all exactly verified, dissonance counters zero.

Scene	DOF	active	leaves	engine	certify (end to end)	verify	ground truth
Planar relay (E3)	2	2	46	0.48 s	—	0.09 s	—
Comb	3	2	54	1.36 s	—	0.17 s	0 / 300k
Shoulder–elbow (anchor)	4	2	8	0.04 s	—	0.04 s	0 / 150k
iiwa-like bin	5	3	8	0.15 s	1.14 s	0.16 s	0 / 40k
iiwa7 shelf (flagship)	7	3	2	1.3 s	46.7 s	2.42 s	0 / 40k + 448 corners
iiwa7 stacked crate	7	3	2	1.3 s	46.1 s	2.43 s	0 / 40k + 448 corners
iiwa7 safety guard	7	3	4	1.6 s	47.1 s	2.50 s	0 / 40k + 448 corners
Sealed wall, $k = 3 \dots 7$	k	k	2–4	0.1–65 s	0.15–80 s	0.01–6.8 s	0 / 8k + 8k corner-biased

column generation, a Kronecker-structured lazy Bernstein basis, or a correlative-sparsity analogue of Waki et al. [39] for Bernstein-LP are candidate levers, the last of which is open.

Formalisation. The Lean theorems are about the checker and the reference definitions; the JSON parser and the compiler stand behind every binary `PROOF` except the one scene evaluated in the kernel, and the kernel evaluation of the unmodified checker does not scale past that scene (§6). A kernel-oriented twin of the checker, proved equal to it, would extend the per-scene theorems to the flagship; the sides-not-points remark of §4 and the $q \leftrightarrow s$ translation are short additions not yet made; prismatic joints are absent from both verifiers.

Toward cages. A separator made of several low-dimensional slab pieces glued by exactly certified overlaps—a slab complex—would address disconnections a single scalar barrier cannot; conceptually it is the triangulated separator of Li and Dantam, made exact. That is a second paper.

10 Provenance and anticipated objections

Provenance. The author takes full responsibility for the content of this paper. The work was carried out with AI coding assistants under human direction: the implementation by an autonomous coding agent working in sessions from a written specification, the sessions supervised and reviewed by a second AI system, and every gate, every scene validation, every design decision and the go/no-go decision signed by the human author. We disclose this for two reasons. First, because it is true. Second, because it explains the design: the verifier is the trust anchor *precisely because* the generation pipeline is not to be trusted—a 499-line exact program a person can read is the only component whose correctness we ask anyone to take on inspection, and its adversarial test suite is the evidence for that. The episodes of §8.7 are the other half of the argument: dense sampling vouched for a disconnection that did not exist—once in a scene we had authored ourselves—and the requirement of a certificate refused it: the engine could not find a witness on every cell, so no certificate was written and the verifier was never reached. The verifier guards the opposite failure, a generator that would write a wrong certificate. Two artefacts that did lie—figures—were caught only by the human gate; we take that as a statement about what tests can and cannot establish, and we state the two properties (fidelity and sufficiency) separately for that reason. The text was checked in the same spirit: every statement about the implementation was confirmed against the source code, every number is read from a dated benchmark file, and every reference was checked against its primary source, which corrected three errors in our own notes (a journal title, an author list and a DOI). Version 2 adds the Lean development, written by the same coding agent in three sessions from a list of nineteen proof obligations; its reference definitions were re-read by the supervising system against the Python verifier and against textbook conventions before anything was claimed about them; its numbers are read from two archived quiet-machine runs of 10 October 2026; and its four prior-work references were checked against primary sources, which corrected an author order and removed a reference that did not exist.

Objections we expect.

"This is 3-DOF in disguise."
The theorem quantifies over the full 7-dimensional joint box; passive dimensions are covered by the proof, not excluded from it (§4). What is 3-dimensional is the cost. How the cost behaves as the active count grows is measured, not assumed: the sealed-wall family of §7.2 raises it from 3 to 7, every instance PROOF and exactly verified, with the per-leaf LP growing as $(d+1)^k$ and a single-LP time frontier at $k = 8$. What remains unmeasured is the regime where many leaves and many active dimensions occur together.

"Rational arithmetic is not the hard part."
Agreed. Finding infeasibility proofs that scale is the hard part, and this paper does not claim to advance it (§7). Rational arithmetic buys neither precision nor scale; it buys a verifier small enough to read, that trusts neither the planner nor the collision checker. The contribution is the independence of the check, not the number system.

"An approximate decomposition must introduce error."
It introduces incompleteness, not error. Cells are exact rational boxes bisected at midpoints; the Bernstein test on a cell is conservative, so a cell it cannot certify is reported undecided and the certificate is simply not issued. The only floating-point computation, the LP solve, searches for a witness; the witness is then rounded to rationals and re-checked exactly. Nothing approximate enters Theorem 1.

"The scenes are chosen to work."
They are—that is §7, stated as scope. The selection was also measured: on the real link, one robust proximal trap exists, and we say so instead of dressing three scenes as three mechanisms.

"Why not rationalize an SOS certificate?"
One can, and the exact-SOS line does (§2). The step that makes it fragile—recovering an exactly positive-semidefinite Gram matrix from a floating-point interior-point solution—is absent from a linear program over Bernstein coefficients: exactness reduces to rounding rationals and re-checking inequalities, which succeeded at the first denominator bound on all seven scenes here. That is what lets the verifier be 499 lines with no linear algebra, and exactness be routine rather than an achievement per instance.

"The iiwa is not exact."
Correct, and neither is its URDF (§4). We certify a rational kinematics that reproduces the published description to $2 \cdot 10^{-6}$ m at the flange.

"The speed comparison is unfair."
We make none (§7).

"A Lean proof only proves what it states."
Exactly, and §6 lists what it states and what it does not: a statement in s-space about one body and the listed closed obstacles, soundness only, with the parser and the compiler trusted behind a binary `PROOF` and dropped for the one scene checked in the kernel. The value is not that nothing is trusted; it is that what is trusted is short, listed, and contains no polynomial.

"Why keep the Python verifier at all?"
Because it is what a user runs without a Lean toolchain, because a person can still read it, and because two independent checkers that agree on 1 061 inputs are better evidence than either alone.

"`native_decide` would have proved every scene."
It would, in seconds, by adding the compiler to the trust base as an axiom per use. We did it, in a file named for it, and the audit refuses that axiom anywhere else.

"The proof of Theorem 1 has a boundary case."
It does not: the proof picks a point where $\varphi = 0$, strictly inside the slab; the outside test is non-strict and correct. An earlier draft of this paper got that wrong, and the code was right; the fact-check against the implementation that found it is part of how the paper was written.

References

[1] Alexandre Amice, Hongkai Dai, Peter Werner, Annan Zhang, and Russ Tedrake. Finding and optimizing certified, collision-free regions in configuration space for robot manipulators. In *Algorithmic Foundations of Robotics XV (WAFR 2022)*, 2022.

[2] Sajid Ansari, Arthi, Girish Varma, and Antony Thomas. Learning motion feasibility from point clouds in cluttered environments, 2026. arXiv:2606.26700.

[3] Stéphane Cambon, Rachid Alami, and Fabien Gravot. A hybrid approach to intricate motion, manipulation and task planning. *The International Journal of Robotics Research*, 28(1):104–126, 2009.

[4] John F. Canny. *The Complexity of Robot Motion Planning*. MIT Press, 1988.

[5] Jose Capco, Mohab Safey El Din, and Josef Schicho. Robots, computer algebra and eight connected components. In *Int. Symp. on Symbolic and Algebraic Computation (ISSAC)*, pages 62–69, 2020.

[6] Jose Capco, Mohab Safey El Din, and Josef Schicho. Positive dimensional parametric polynomial systems, connectivity queries and applications in robotics. *Journal of Symbolic Computation*, 115:320–345, 2023.

[7] Hongkai Dai, Alexandre Amice, Peter Werner, Annan Zhang, and Russ Tedrake. Certified polyhedral decompositions of collision-free configuration space. *The International Journal of Robotics Research*, 2024. DOI 10.1177/02783649231201437; arXiv:2302.12219.

[8] Neil T. Dantam, Zachary K. Kingston, Swarat Chaudhuri, and Lydia E. Kavraki. An incremental constraint-based framework for task and motion planning. *The International Journal of Robotics Research*, 37(10):1134–1151, 2018.

[9] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*. Springer, 2021. DOI 10.1007/978-3-030-79876-5_37.

[10] Jürgen Garloff. Convergent bounds for the range of multivariate polynomials. In *Interval Mathematics 1985*, volume 212 of *LNCS*, pages 37–56, 1985.

[11] Caelan Reed Garrett, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. FFRob: Leveraging symbolic planning for efficient task and motion planning. *The International Journal of Robotics Research*, 37(1):104–136, 2018.

[12] Didier Henrion. Positively not SOS: pseudo-moments and extreme rays in exact arithmetic, 2025. arXiv:2509.01382.

[13] Didier Henrion. Solving Stengle's example in rational arithmetic: exact values of the moment-SOS relaxations, 2025. arXiv:2512.19141.

[14] Didier Henrion, Jared Miller, and Mohab Safey El Din. Algebraic proofs of path disconnectedness using time-dependent barrier functions, 2024. arXiv:2404.06985. Preprint as of September 2026. Code: github.com/jarmill/set_connected.

[15] Qi Huangfu and J. A. Julian Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

[16] Luc Jaulin. Path planning using intervals and graphs. *Reliable Computing*, 7(1):1–15, 2001.

[17] Erich Kaltofen, Bin Li, Zhengfeng Yang, and Lihong Zhi. Exact certification of global optimality of approximate factorizations via rationalizing sums-of-squares with floating point scalars. In *Int. Symp. on Symbolic and Algebraic Computation (ISSAC)*, pages 155–164, 2008.

[18] Sihui Li and Neil T. Dantam. Towards general infeasibility proofs in motion planning. In *IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, pages 6704–6710, 2020.

[19] Sihui Li and Neil T. Dantam. Learning proofs of motion planning infeasibility. In *Robotics: Science and Systems XVII*, 2021. Paper 64, DOI 10.15607/RSS.2021.XVII.064.

[20] Sihui Li and Neil T. Dantam. Exponential convergence of infeasibility proofs for kinematic motion planning. In *Algorithmic Foundations of Robotics XV (WAFR 2022)*, 2022. DOI 10.1007/978-3-031-21090-7_18.

[21] Sihui Li and Neil T. Dantam. A sampling and learning framework to prove motion planning infeasibility. *The International Journal of Robotics Research*, 42(10):938–956, 2023. DOI 10.1177/02783649231154674.

[22] Sihui Li and Neil T. Dantam. Scaling infeasibility proofs via concurrent, codimension-one, locally-updated Coxeter triangulation. *IEEE Robotics and Automation Letters*, 8(12):8303–8310, 2023.

[23] Sihui Li and Neil T. Dantam. Scaling motion planning infeasibility proofs, 2024. arXiv:2406.04795. Preprint as of September 2026.

[24] Victor Magron and Mohab Safey El Din. On exact Polya and Putinar's representations. In *Int. Symp. on Symbolic and Algebraic Computation (ISSAC)*, 2018.

[25] Victor Magron and Mohab Safey El Din. RealCertify: a Maple package for certifying non-negativity. *ACM Communications in Computer Algebra*, 52(2):34–37, 2018.

[26] Victor Magron and Mohab Safey El Din. On exact Reznick, Hilbert-Artin and Putinar's representations. *Journal of Symbolic Computation*, 107:221–250, 2021. DOI 10.1016/j.jsc.2021.03.005.

[27] Victor Magron, Xavier Allamigeon, Stéphane Gaubert, and Benjamin Werner. Formal proofs for nonlinear optimization. *Journal of Formalized Reasoning*, 8(1):1–24, 2015. DOI 10.6092/issn.1972-5787/4319.

[28] Érik Martin-Dorel and Pierre Roux. A reflexive tactic for polynomial positivity using numerical solvers and floating-point computations. In *Proc. 6th ACM SIGPLAN Conference on Certified Programs and Proofs (CPP 2017)*, pages 90–99, 2017. DOI 10.1145/3018610.3018622.

[29] Zoe McCarthy, Timothy Bretl, and Seth Hutchinson. Proving path non-existence using sampling and alpha shapes. In *IEEE Int. Conf. on Robotics and Automation (ICRA)*, pages 2563–2569, 2012.

[30] César Muñoz and Anthony Narkawicz. Formalization of Bernstein polynomials and applications to global optimization. *Journal of Automated Reasoning*, 51(2):151–196, 2013. DOI 10.1007/s10817-012-9256-3.

[31] Helfried Peyrl and Pablo A. Parrilo. Computing sum of squares decompositions with rational coefficients. *Theoretical Computer Science*, 409(2):269–281, 2008.

[32] Sven Polak. Sum-of-squares certificates for symmetric polynomials on the hypercube: a counterexample to a conjecture of De Klerk and Laurent, 2026. arXiv:2605.31169. Exact rational certificate with archived verification scripts.

[33] Mihai Putinar. Positive polynomials on compact semialgebraic sets. *Indiana University Mathematics Journal*, 42(3):969–984, 1993.

[34] Alexey Solovyev and Thomas C. Hales. Formal verification of nonlinear inequalities with Taylor interval approximations. In *NASA Formal Methods (NFM 2013)*, volume 7871 of *Lecture Notes in Computer Science*, pages 383–397. Springer, 2013. DOI 10.1007/978-3-642-38088-4_26.

[35] Russ Tedrake and the Drake Development Team. Drake: Model-based design and verification for robotics. `https://drake.mit.edu`, 2019.

[36] The mathlib Community. The Lean mathematical library. In *Proc. 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020. DOI 10.1145/3372885.3373824.

[37] Antony Thomas, Fulvio Mastrogiovanni, and Marco Baglietto. An incremental sampling and segmentation-based approach for motion planning infeasibility. *Robotics and Autonomous Systems*, 2025. arXiv:2501.11434.

[38] Anastasiia Varava, J. Frederico Carvalho, Danica Kragic, and Florian T. Pokorny. Free space of rigid objects: Caging, path non-existence, and narrow passage detection. *The International Journal of Robotics Research*, 40 (10–11), 2021.

[39] Hayato Waki, Sunyoung Kim, Masakazu Kojima, and Masakazu Muramatsu. Sums of squares and semidefinite program relaxations for polynomial optimization problems with structured sparsity. *SIAM Journal on Optimization*, 17(1):218–242, 2006.

[40] David Wang and Mohammad Abdulaziz. Formally verified certification of unsolvability of temporal planning problems, 2025. arXiv:2510.10189. Isabelle/HOL; AI temporal planning, not geometric motion planning.

[41] Chee-Keng Yap. Towards exact geometric computation. *Computational Geometry: Theory and Applications*, 7 (1–2):3–23, 1997.

[42] Chee-Keng Yap and Thomas Dubé. The exact computation paradigm. In Ding-Zhu Du and Frank Hwang, editors, *Computing in Euclidean Geometry*, pages 452–492. World Scientific, 2nd edition, 1995.

[43] Liangjun Zhang, Young J. Kim, and Dinesh Manocha. Efficient cell labelling and path non-existence computation using C-obstacle query. *The International Journal of Robotics Research*, 27(11–12):1246–1257, 2008.